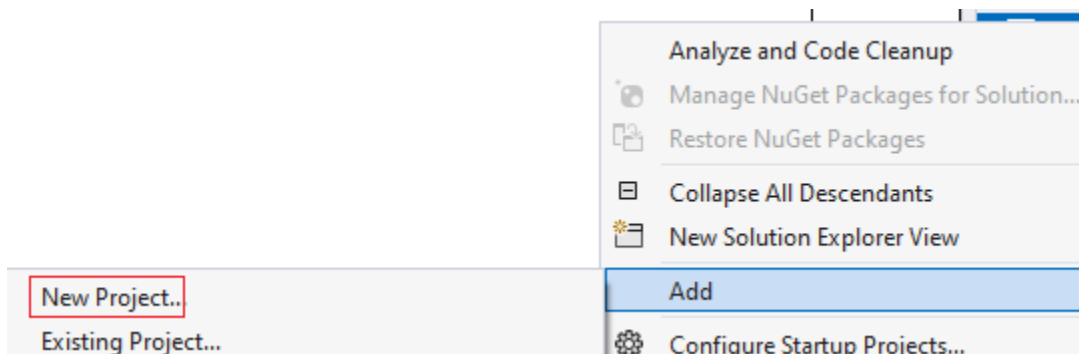


ПОДГОТОВКА ЗА ИЗПИТВАНЕ



Additional information

Console App

C#

Linux

macOS

Windows

Console

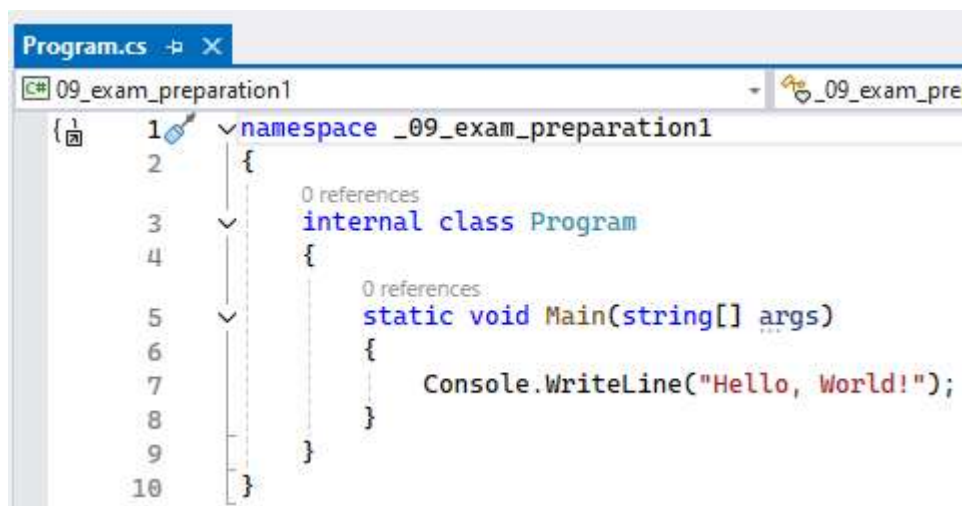
Framework ⓘ

.NET 8.0 (Long Term Support)

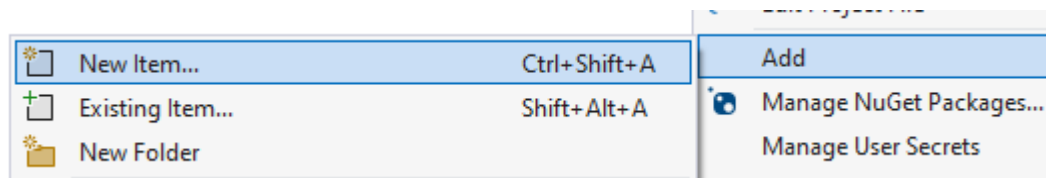
☒ Do not use top-level statements ⓘ

☐ Enable native AOT publish ⓘ

top-level statement в
.NET 8.0 скриват
class Program + Main()



Ако имате if-else и други сложни конструкции, трябва за всеки възможен изход да има return!

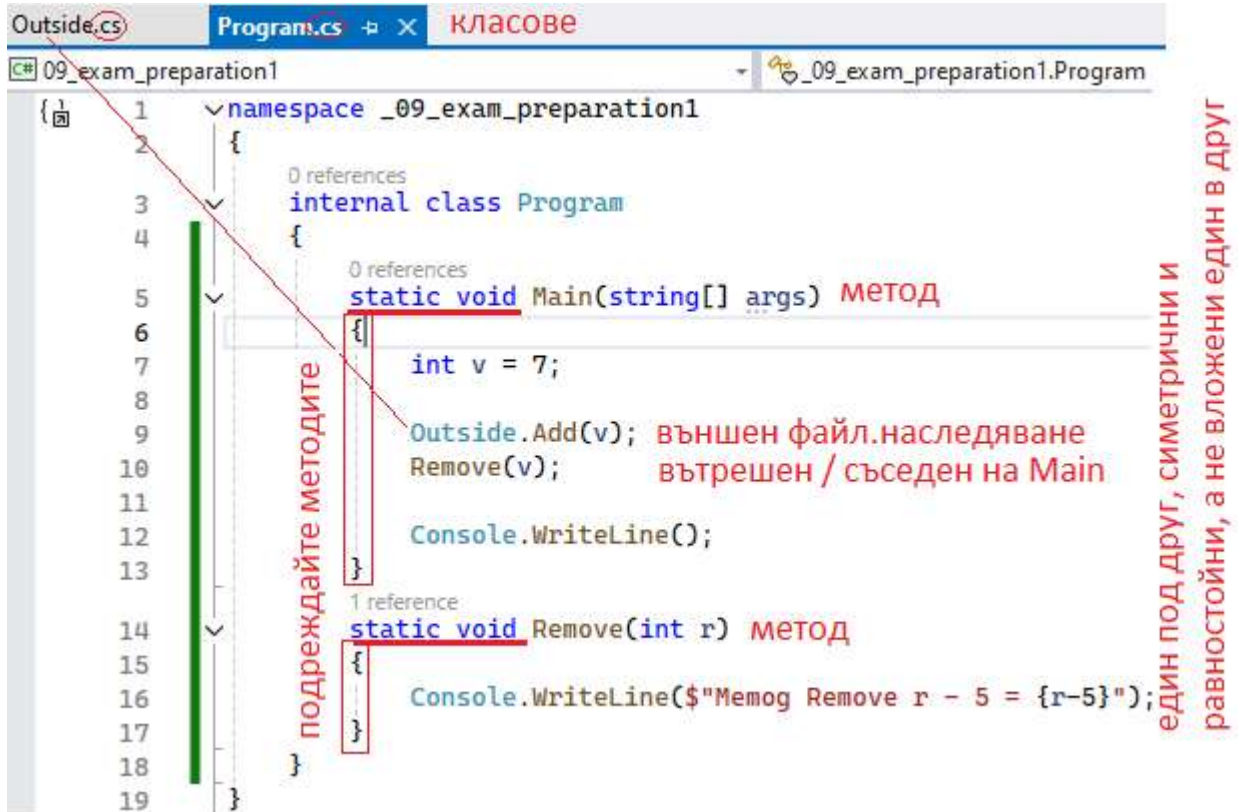


Нека създадем нов клас Outside.cs.

Outside.cs* Когато редактирате файл, обръщайте внимание дали има знак * до името. Той показва, че има промени, които не са запазени. Просто натиснете **Ctrl + S** = Save, ако не се намирате в Program.cs (само тя се запазва автоматично при всеки RUN). Така промените могат да се отразят след редакция при стартиране на основния Program.cs!!!

external class Outside
This method has 1 reference(s). (Alt+2)
1 reference
public static void Add(int a)

- Преди да изтриете метод, уверете се, че има 0 референции към него. Не е добре да изтривате метод, който ще бъде извикан някъде.



!!! Main() – методът е private!!! Когато не пише нищо, по подразбиране е private. Main() е private, защото се извиква от runtime, а не от потребителския код на други класове. Ако Main() е public, ние ще можем да го извикваме от други файлове с класове и методи, и тогава какво би се случило? Входната точка на програмата да бъде извиквана? Има опасност началото да се извиква наново и наново. Принципно е възможно ръчно да зададем Main() да бъде public static и да бъде извикван от други класове, но така подредбата ще се разпръсне, защото казахме, че Main() е входната точка на програмата, началото, като съдържание на книга. Runtime винаги намира и извиква Main() независимо дали е private или public. Не правете Main() public без особена причина! Когато Main() е защитен да бъде private (защитен и скрит), се постига по-добра енкапсулация на вътрешната логика.

Запомнете: във всички методи, локалните параметри и променливи се пишат camelCase.

- **Private - camelCase:** параметри, локални променливи, полета в класове обикновено с _представка: `_firstName`,
- **Public - PascalCase:** свойства, методи, класове, интерфейси

```

1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace _09_exam_preparation1
8  {
9      internal class Outside    клас.метод
10     {
11         public static void Add(int a)
12         {
13             Internal();
14             Console.WriteLine($"Това е метод Add, a+2 = {a+2}"); ;
15         }
16
17         private static void Internal()
18         {
19             Console.WriteLine("Вложеният метод Internal, който е private");
20         }
21     }
22 }

```

```

Microsoft Visual Studio Debug Console
Вложеният метод Internal, който е private
Това е метод Add, a+2 = 9
Метод Remove r - 5 = 2

D:\Exercises\11 class\09_exam_preparation1\
Press any key to close this window . . .

```

Операнд – знак за операция +, -, *, /, % ...
 префикс ++a, --a (знакът за операция е ПРЕди променливата)
 постфикс a++, a--
 инфикс a – b
 a, b – операнди
 a + b – бинарна операция (би – 2 операнда)
 -a – унарна операция (само с 1 операнд).
 m / p – мантиса / порядък
 Дробни константи могат да използват буквата E за означаване * 10,
 например 3.14 = 0.314E1 = 314E-2
 ++ - инкрементиране (нараства)
 -- - декрементиране (намалява)
 using - включва библиотека (колекция) от вградени методи

```

// Цели числа
byte b = 255;           // 0 go 255 (1 байт)
sbyte sb = -128;        // -128 go 127 (1 байт)
short s = -32768;        // -32,768 go 32,767 (2 байта)
ushort us = 65535;       // 0 go 65,535 (2 байта)
int i = 2147483647;      // -2.1e9 go 2.1e9 (4 байта)
uint ui = 4294967295;    // 0 go 4.3e9 (4 байта)
long l = 9223372036854775807; // -9.2e18 go 9.2e18 (8 байта)
ulong ul = 18446744073709551615; // 0 go 1.8e19 (8 байта)

// Числа с плаваща запетая
float f = 3.14f;         // ~6-9 цифри (4 байта)
double d = 3.1415926535; // ~15-17 цифри (8 байта)
decimal dec = 123.456m;  // 28-29 значими цифри (16 байта)

// Други
char c = 'A';            // Unicode символ (2 байта)
bool flag = true;        // true/false (1 байт)
string str = "Hello C#"; // непроменлив низ
object obj = "Hello World"; // базов клас за всички типове

// Други
char c = 'A';            // Unicode символ (2 байта)
bool flag = true;        // true/false (1 байт)
string str = "Hello C#"; // непроменлив низ
object obj = "Hello World"; // базов клас за всички типове
//-----
// Nullable
int? nullableInt = null; // ? позволява да съдържа null
Nullable<double> nullableDouble = 3.14;
DateTime? date = DateTime.Now;
//-----
// var variable - автоматично се разпознава, но трябва да е вътре в метод
var name = "John"; // string
var age = 25;      // int
//-----
// динамичен тип - името се запазва, но типът се променя - също трябва да е в метод
dynamic value = 10;
value = "Hello"; // Валидно

```



```

// Масиви - от прости еднотипни данни
int[] numbers = new int[5] { 1, 2, 3, 4, 5 };
string[] names = new string[3];
//-----
//Персонален тип enum - ние определяме какво да съдържа
2 references
enum Days { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday }
Days today = Days.Monday;
//-----
// Сметков mun
2 references
struct Point
{
    public int X;
    public int Y;
}

Point p = new Point { X = 10, Y = 20 };
//-----
// Класове - от различни типове данни по шаблон на обект
2 references
class Person
{
    public string Name;
    public int Age;
}

Person person = new Person { Name = "John", Age = 30 };

// Интерфейси - започват с I, наследяват клас
0 references
class Dog : IAnimal
{
    0 references
    public void MakeSound() => Console.WriteLine("Woof!");
}
//-----
// Делегати
delegate void MyDelegate(string message);
MyDelegate del = message => Console.WriteLine(message);

```

ЗАДАЧИ ЗА ПОДГОТОВКА

1 зад: Потребителят въвежда начално и крайно число, а на конзолата се отпечатват всички числа между тях.

```
static void PrintNumbers(int start, int end)
{
    for (int i = start; i <= end; i++)
    {
        Console.Write("{0} ", i);
    }
}
```

2 зад: Потребителят въвежда поотделно име и фамилия, а метод връща като резултат пълното име.

```
static string ReadFullName()
{
    string firstName = Console.ReadLine();
    string lastName = Console.ReadLine();
    return firstName + " " + lastName;
}
```

3 зад: Създайте метод за повдигане на число на степен:

```
static double MathPower(double number, int power)
{
    double result = 1;
    for (int i = 0; i < power; i++)
        result *= number;
    return result;
}
```

4 зад: Да се въведе число и да се отпечата матрица (запълнен квадрат) от въведеното число

```
0 references
static void Main(string[] args)
{
    int n = int.Parse(Console.ReadLine());

    NxNMatrix(n);
}

1 reference
static void NxNMatrix(int n)
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            Console.Write($"{n} ");
        }
        Console.WriteLine();
    }
}
```

5 зад: Да се въведе редица от числа, разделени с интервал и изведе сумата от четните.

```
Console.Write("Въведете редица от числа: ");
int[] numbers = Console.ReadLine().Split(' ').Select(int.Parse).ToArray();

int sum = 0;

for (int i = 0; i < numbers.Length; i++)
{
    int currentNum = numbers[i];
    if (currentNum % 2 == 0)
        sum += currentNum;
}

Console.WriteLine($"Сумата на четните числа е: {sum}");
```


6 зад: Да се въведат определен брой числа – всяко на нов ред и отпечатат в обратен ред:

```
Console.Write("Въведете брой редове: ");
int lines = int.Parse(Console.ReadLine());
Console.WriteLine("Въведете всяко число на нов ред: ");

int[] numbers = new int[lines];

for (int i = 0; i < lines; i++)
{
    int n = int.Parse(Console.ReadLine());
    numbers[i] = n;
}

for (int i = numbers.Length - 1; i >= 0; i--)
{
    Console.Write(numbers[i] + " ");
}
```

7 зад: Да се сравнят 2 редици и ако са различни, да се отпечата в коя позиция, а ако съвпадат – да се изведе сумата от числата в редицата.

```
int[] arr1 = Console.ReadLine().Split().Select(int.Parse).ToArray();
int[] arr2 = Console.ReadLine().Split().Select(int.Parse).ToArray();

if (arr1.Length != arr2.Length)
    Console.WriteLine($"Редиците са с различна дължина!");

int sum = 0;

for (int i = 0; i < arr1.Length; i++)
{
    if (arr1[i] != arr2[i])
    {
        Console.WriteLine($"Редиците са различни в позиция {i}.");
        return;
    }
    else
        sum += arr1[i];
}

Console.WriteLine($"Редиците са идентични. Тяхната сума е: {sum}");
```

8 Зад: Да се въведе поредица от символи, разделени с интервал и отпечата в обратен ред:

```
Console.WriteLine("Въведете поредица: ");
var items = Console.ReadLine().Split(' ').ToArray();

for (int i = 0; i < items.Length / 2; i++)
{
    var temp = items[i];
    items[i] = items[items.Length - 1 - i];
    items[items.Length - 1 - i] = temp;
}

Console.WriteLine(string.Join(" ", items));
```

9 зад: Да се преброят гласните в дума:

```
static void Main(string[] args)
{
    Console.WriteLine("Enter a word: ");
    string input = Console.ReadLine().ToLower();

    Console.WriteLine($"The vowels are {VowelCounter(input)}");
}

1 reference
static int VowelCounter(string word)
{
    int counter = 0;

    for (int i = 0; i < word.Length; i++)
    {
        switch (word[i])
        {
            case 'a':
            case 'e':
            case 'i':
            case 'o':
            case 'u':
            case 'y': counter++; break;
        }
    }

    return counter;
}
```

10 зад: Въведете число и да се изпише дали е то палиндром или не (симетрично спрямо средата).

```
static void Main(string[] args)
{
    Console.Write("Enter a number to check is it a palindrome: ");
    string input = Console.ReadLine();

    while (input != "END")
    {
        Palindrome(input);

        input = Console.ReadLine();
    }
}

1 reference
static void Palindrome(string input)
{
    bool isPalindrome = false;

    for (int i = 0; i < input.Length / 2; i++)
    {
        if (input[i] != input[input.Length - i - 1])
        {
            isPalindrome = false;
        }
        else
            isPalindrome = true;
    }
    Console.WriteLine(isPalindrome.ToString().ToLower());
}
```